

Flutter Profile Card

Project Documentation

Problem Statement: Build a Flutter profile card screen using Column, Row, Container, CircleAvatar, Text, and Icon widgets with custom theme colors.

1. Introduction

This project implements a complete Flutter profile card/profile screen with a custom visual theme, responsive layout, reusable widgets, and basic user interactions. The implementation is split between `main.dart`, which starts the application and configures the Material theme, and `profile_card.dart`, which contains the design tokens, profile data, screen layout, interactions, animations, and reusable UI widgets.

Although the assignment focuses on fundamental Flutter widgets, the implementation demonstrates how those widgets can be combined into a structured interface rather than being used in isolation. The screen uses Column for vertical structure, Row for horizontal alignment, Container for styling and separators, CircleAvatar for the profile image area, Text for information, and Icon for visual actions and metadata.

2. Objectives

- Create a profile card/profile screen using core Flutter layout and display widgets.
- Apply a custom color palette and typography instead of relying entirely on Material defaults.
- Build a responsive layout that changes between stacked and two-column arrangements.
- Add basic interactions such as Follow, Message, Share, More Options, and activity details.
- Use reusable widgets and helper methods to keep the UI organized.
- Demonstrate state management, animations, bottom sheets, and feedback using SnackBar.

3. Project Structure

`main.dart`

- Imports Flutter Material and `profile_card.dart`.
- Defines `ProfileApp` as the root `StatelessWidget`.
- Creates `MaterialApp` with the application title, theme, and `ProfileScreen` as home.
- Builds a custom `ThemeData` configuration.

`profile_card.dart`

- Defines `AppColors`, `AppSpacing`, and `AppText` design tokens.
- Stores profile and activity data.
- Implements `ProfileScreen` and its state.
- Handles follow state, animations, bottom sheets, and SnackBar feedback.
- Contains reusable widgets such as `_Pressable`, `_FollowButton`, `_IconActionButton`, `_Stat`, `_SectionHeading`, `_DetailRow`, `_TimelineItem`, `_SheetRow`, and `_SheetHandle`.

4. Application Entry Point – main.dart

The main.dart file is intentionally small. The main() function calls runApp(const ProfileApp()), which places the root widget into the Flutter widget tree. ProfileApp extends StatelessWidget because the application-level configuration itself does not need mutable state.

MaterialApp is configured with the title 'Profile', debugShowCheckedModeBanner disabled, a custom theme, and ProfileScreen as the initial screen. The _buildTheme() method creates a Material 3 base theme and overrides the color scheme and component styles to maintain the application's custom visual identity.

5. Custom Theme and Design Tokens

The project uses a deliberate palette defined in AppColors. The main background is a warm bone/canvas color, the primary text uses near-black ink, secondary information uses muted stone, and oxblood is used as the single accent. This makes the interface visually consistent.

Token	Purpose	Value
canvas	Page background	#F3F0E9
surface	Sheets and input fills	#FAF8F3
ink	Primary text / main actions	#17171A
stone	Secondary and metadata text	#8C8579
hairline	Borders and dividers	#DCD5C6
accent	Oxblood accent	#7A2A2F
accentTint	Avatar background	#F1DEDD

AppSpacing provides a consistent scale from xs (4) through xxxl (32). AppText defines explicit text styles for the hero name, role, metadata, body text, section titles, statistics, details, navigation labels, sheet titles, and rows. Separating these values from individual widgets makes the visual system easier to maintain.

6. Main Profile Screen

ProfileScreen is a StatefulWidget because the Follow/Following state changes during interaction and the entrance animation updates the UI after the first frame. The screen stores fictional profile information including the name, username, role, location, biography, project count, following count, follower count, phone number, and three recent activity entries.

The profile identity is displayed through a header containing a CircleAvatar with the initials MR. A Container creates the oxblood circular ring around the avatar. Text widgets display the name, username, role, location, and biography.

7. Required Flutter Widgets Used

Widget	Use in Project
Column	Provides the main vertical structure, including the screen content, identity section, About section, Activity section, statistics, and bottom-sheet content.
Row	Arranges the top bar controls, avatar and identity information, action buttons, statistics, section headings, and detail rows horizontally.

Container	Provides borders, backgrounds, spacing-related visual elements, avatar ring, timeline markers, vertical rules, and button surfaces.
CircleAvatar	Displays the profile avatar area using initials 'MR' with a custom background color.
Text	Displays profile information, labels, statistics, activity titles, messages, and other content.
Icon	Provides recognizable visual markers for actions, location, role, activity, sharing, messaging, and settings.

8. Responsive Layout

The screen uses `LayoutBuilder` to inspect available width. When the width is at least 760 pixels, the identity section and details section are displayed side by side in a `Row`. A vertical `Container` acts as a dividing rule. On narrower screens, the same content is arranged vertically in a `Column` with a horizontal `Divider` between major sections.

The content width is constrained to 1040 pixels on wide layouts and 640 pixels on narrow layouts. This prevents the interface from becoming excessively stretched while still allowing it to adapt to different screen sizes.

9. Interaction and State

The Follow button uses `_isFollowing` as mutable state. `_toggleFollow()` changes the state using `setState()` and increments or decrements the follower count. `_formatCount()` converts counts of 1000 or more into a compact K format.

The Message action opens a modal bottom sheet containing a `TextField` and Send button. The Share action opens a bottom sheet with options to copy the profile link or show a QR code. The More Options action displays Edit profile, Notifications, Privacy, and About this app. These example actions provide feedback through `SnackBar` messages.

Activity entries are tappable. Selecting an activity opens a detail bottom sheet containing its time, title, full description, and a Close button.

10. Animation and Reusable Interaction

The identity block uses `FadeTransition` and `SlideTransition` controlled by an `AnimationController` with a 600 ms duration. The animation starts in `initState()` and is properly released in `dispose()`, which is important for resource management.

`_Pressable` is a reusable `StatefulWidget` that combines `MouseRegion`, `GestureDetector`, `Semantics`, and `AnimatedScale`. It provides hover feedback, press feedback, click behavior, and a semantic label. This same primitive is reused by the Follow button, secondary icon buttons, sheet rows, and timeline items, giving interactions a consistent feel.

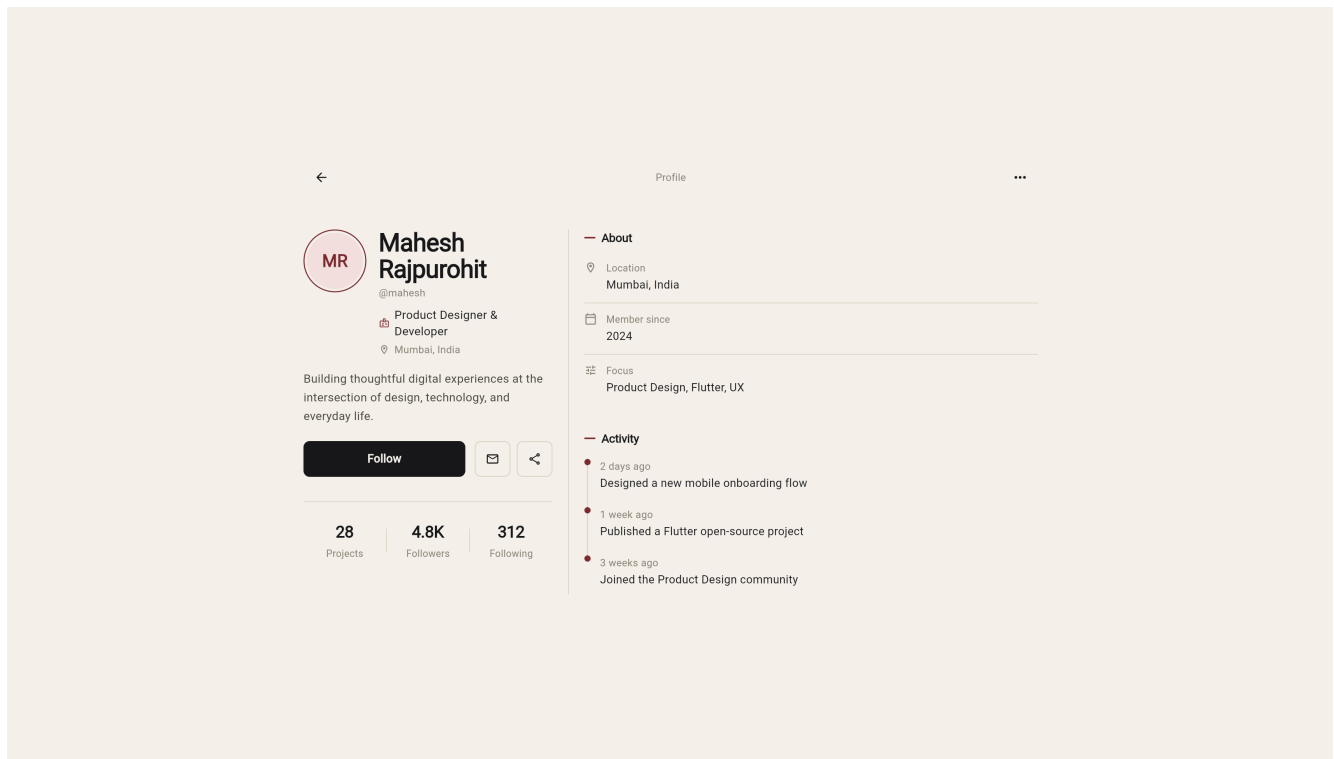
11. About and Activity Sections

The About section presents Location, Member since, and Focus using `_DetailRow`. Each row combines an Icon with a label/value Column and is separated by hairline dividers. The Activity section uses `_TimelineItem` to create a simple timeline with an accent dot and connecting line. Tapping an entry opens its detailed description.

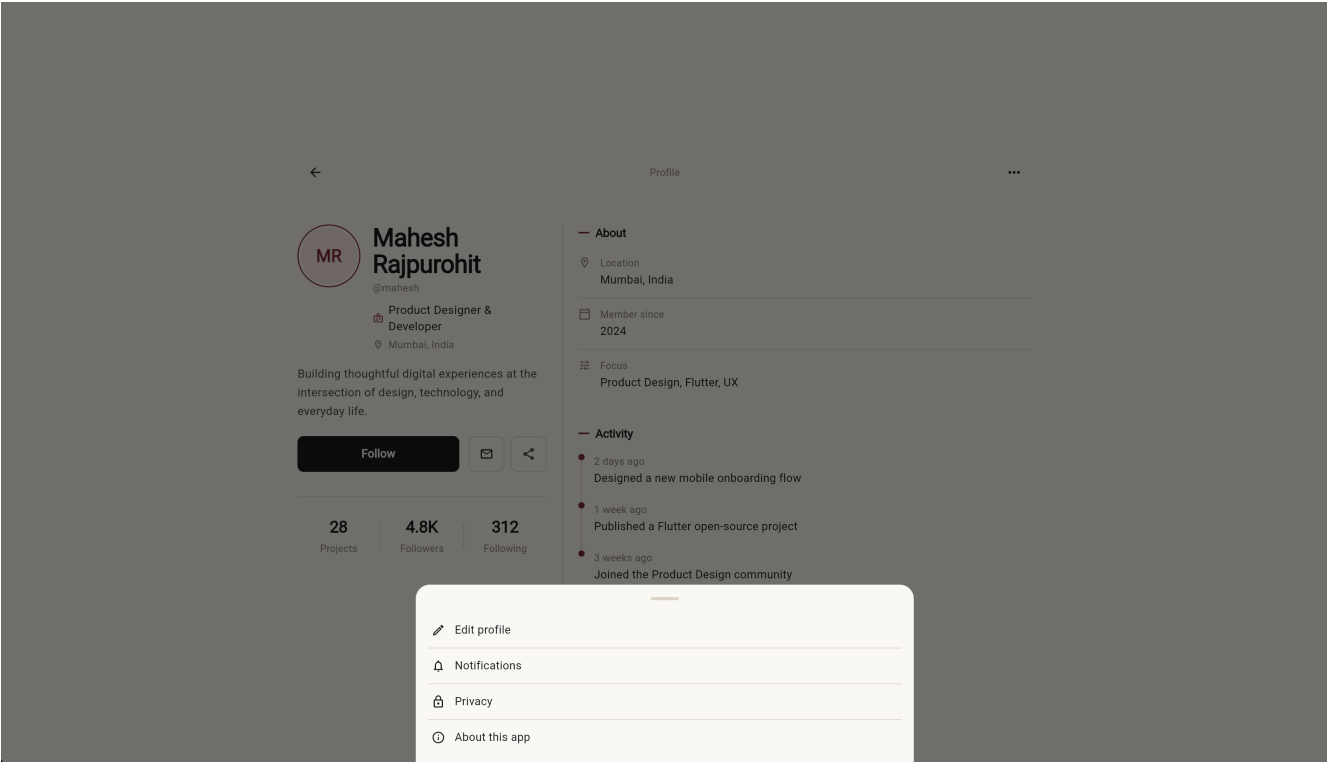
12. Screenshot Documentation

Insert screenshots from the running Flutter application in the spaces below. The suggested screenshots correspond to the main states and interactions demonstrated by the supplied implementation.

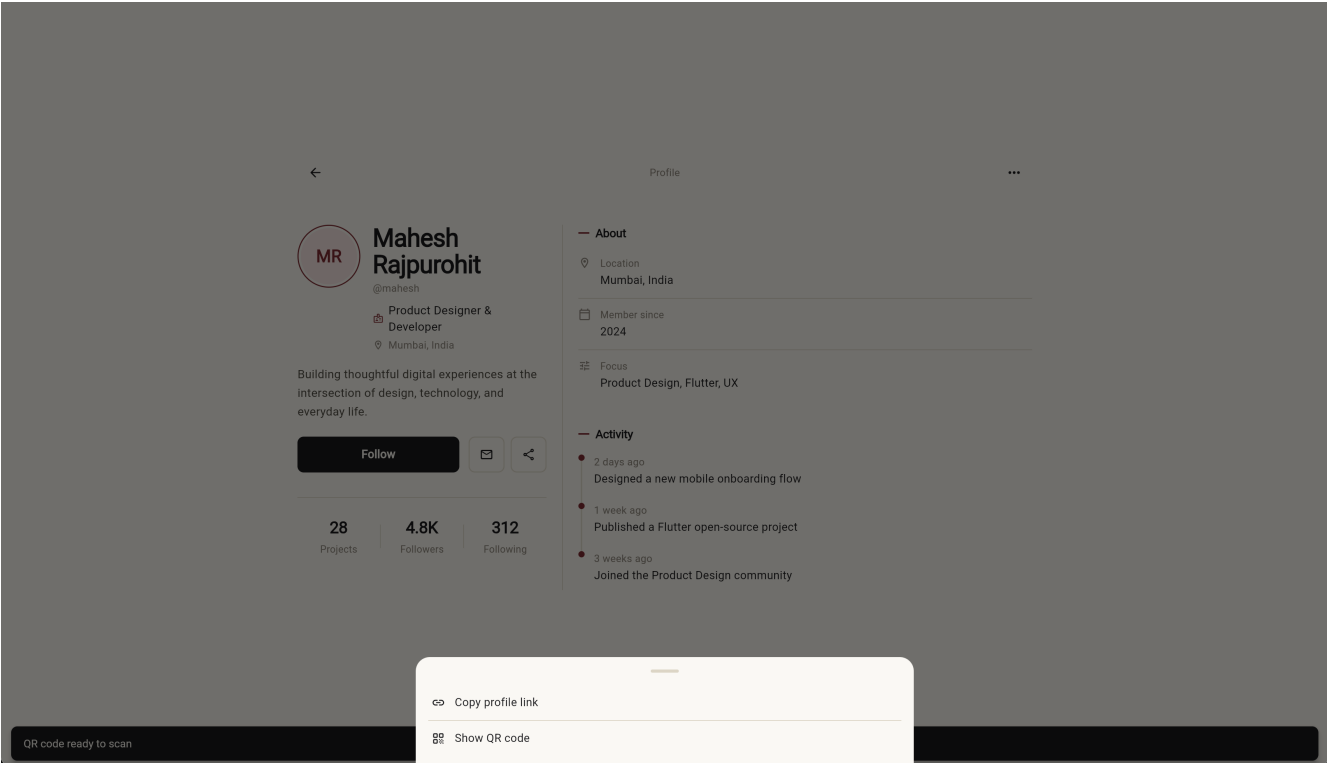
Screenshot 1 – Home/Profile Screen



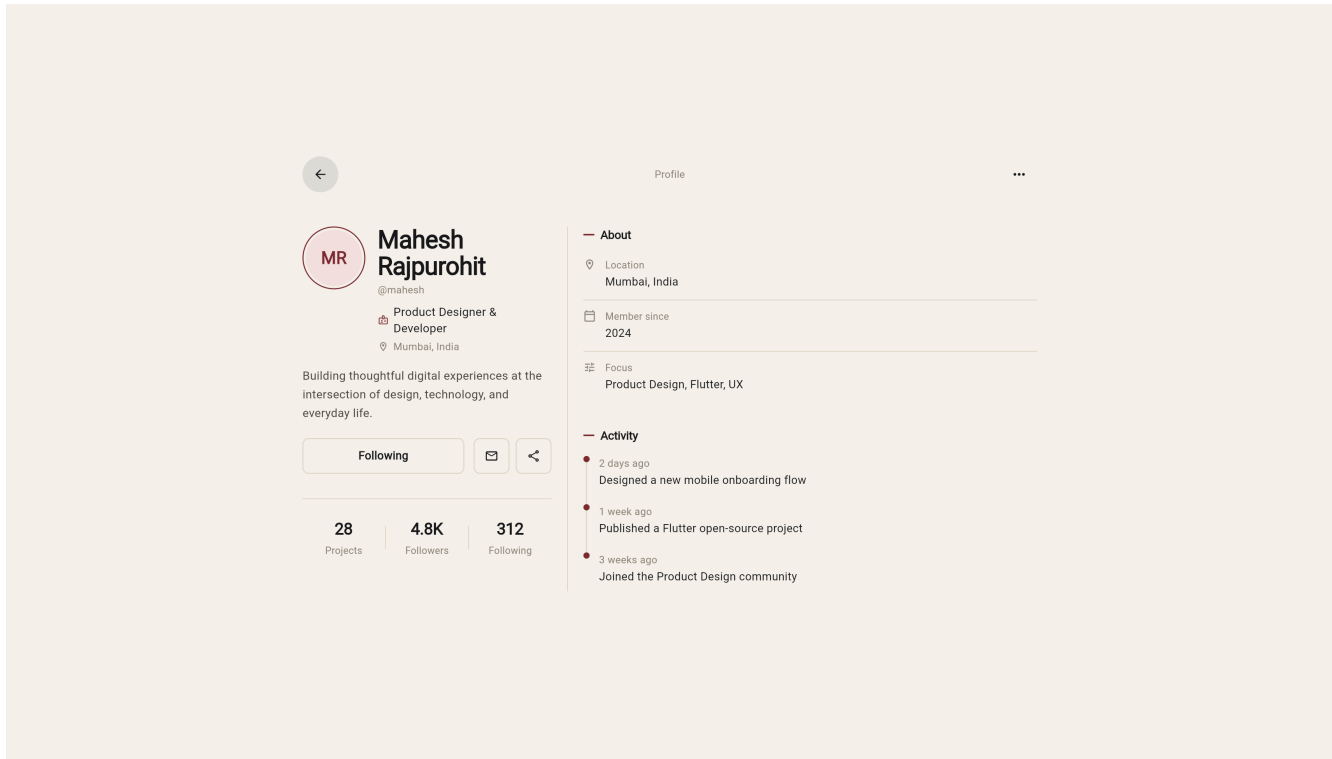
Screenshot 2 – More Options



Screenshot 3 – Share Options



Screenshot 4 – Follow / Activity Interaction



13. Testing and Expected Behavior

- The application should launch directly to the ProfileScreen.
- The profile should remain scrollable on smaller screens.
- Tapping Follow should switch between Follow and Following and update the follower count.
- Tapping Message should open the message composer bottom sheet.
- Tapping Share should display the two share-related options.
- Tapping More Options should display the four available profile options.
- Tapping an activity should open its detail sheet.
- Buttons and interactive rows should provide press/hover feedback where supported.
- The layout should switch between stacked and two-column presentation based on screen width.

14. Conclusion

The completed Flutter project satisfies the stated profile-card problem by using the required core widgets and custom theme colors while extending the screen with responsive layout, state, animations, reusable components, bottom sheets, and user feedback. The separation of design tokens and reusable widgets makes the implementation structured and easier to maintain.

15. What I Learned From This Project

This project helped me understand that building a Flutter interface is not only about placing widgets on the screen. The more important lesson was learning how different parts of a Flutter application work together to create a complete and maintainable user interface. Starting from a simple profile-card requirement, I learned how to organize a screen, control visual consistency, respond to user actions, and make the same design work across different screen sizes.

Understanding Flutter's Widget-Based Approach

One of the most important things I learned was how Flutter represents an interface as a hierarchy of widgets. A screen is built by combining smaller widgets rather than treating the entire interface as one object. Column and Row helped me understand the two most common directions of layout. Column allowed me to build the page vertically, while Row allowed me to place related elements beside one another. Container gave me a way to control decoration, borders, sizes, and backgrounds. Text and Icon provided the actual information and visual indicators. CircleAvatar showed how Flutter can provide a specialized widget for a common interface requirement.

I also learned that widget choice should be based on responsibility. Instead of using one large build method for everything, the project breaks the interface into methods and reusable widgets. This makes it easier to understand what each section does and reduces the difficulty of changing one part of the screen without affecting everything else.

Learning State and User Interaction

Another major lesson was understanding why StatefulWidget is necessary when the interface contains changing information. The Follow button is a simple example: its label changes from Follow to Following, and the follower count changes at the same time. Using setState() tells Flutter that the relevant part of the interface needs to be rebuilt with the new values.

The project also taught me that interaction is more than changing a variable. A good interface should communicate what happened. The project uses Snackbar messages after several actions and uses bottom sheets for secondary tasks. I learned how showModalBottomSheet can present temporary content without navigating away from the main screen. The Message sheet also introduced a TextEditingController and TextField, which demonstrated how user input can be collected before an action is completed.

Responsive Design and Layout Thinking

I learned that a UI should not be designed only for one fixed screen size. The use of LayoutBuilder made it possible to detect available width and change the composition. On a wide viewport, the identity and details areas can sit next to each other. On a narrow viewport, they stack vertically. This showed me that responsive design is not simply shrinking every element; it can require changing the arrangement of content.

Working with constraints also helped me understand why unrestricted content can become difficult to read on large displays. The project uses maximum content widths and different horizontal padding values so the profile remains visually controlled. This is a useful lesson for future applications because the same principle applies to dashboards, forms, portfolio pages, and other interfaces that need to work on phones, tablets, and desktop-sized windows.

Design Systems and Code Organization

The project taught me the practical value of design tokens. Instead of repeatedly writing different colors, spacing values, and text styles throughout the code, AppColors, AppSpacing, and AppText centralize those decisions. This makes the design more consistent and makes future changes faster. For example, changing the accent color in one place can update every component that uses that token.

I also learned about reusable widgets. Components such as `_FollowButton`, `_IconButton`, `_DetailRow`, and `_TimelineItem` each have a focused responsibility. The `_Pressable` widget was especially useful because the same interaction behavior could be shared by multiple controls. This reduced duplication and demonstrated a basic component-oriented approach to Flutter UI development.

Animation, Accessibility, and Resource Management

The entrance animation introduced me to `AnimationController`, `CurvedAnimation`, `FadeTransition`, and `SlideTransition`. I learned that animations need a lifecycle: the controller is initialized in `initState()`, started after initialization, and disposed in `dispose()`. This helped me understand that Flutter widgets can have resources that must be managed correctly.

The project also introduced accessibility-oriented practices. Interactive elements use semantic labels, and tooltips are provided for icon actions. I learned that an interface should not depend only on visual appearance to explain an action. Meaningful labels can help users understand controls and can improve how assistive technologies interpret the interface.

Problem-Solving and Debugging

Another thing I learned from this project was how to approach a user interface as a series of smaller problems instead of trying to build everything at once. Each section of the profile screen has its own purpose, such as the header, actions, statistics, About section, and Activity section. Breaking the screen into these smaller parts made the overall implementation easier to understand and work with.

I also learned that debugging a Flutter interface requires looking at both the widget structure and the state of the application. A problem with alignment, spacing, or interaction may not always come from the widget that is immediately visible. Understanding how parent widgets such as `Column`, `Row`, `Expanded`, and `ConstrainedBox` affect their children helped me think more carefully about where layout problems originate.

Overall Learning Outcome

Overall, this project changed my understanding of a Flutter screen from a collection of widgets into a coordinated system of layout, data, state, interaction, design, and lifecycle management. I learned the fundamentals required to construct a professional-looking screen while still keeping the implementation organized. The experience gives me a stronger foundation for developing larger Flutter applications because the same concepts—widget composition, state updates, responsive layouts, reusable components, theme management, interaction feedback, and lifecycle handling—can be applied repeatedly in more complex projects.